HOOK BACKEND · DELIVERY · 31 OCTOBER 2026

Project delivery

The Hook backend has been moved to its own server, made production-ready and tested at high traffic. All work is complete, merged into the main branch and live.

Delivered

ITEM	DESCRIPTION
Server	Dedicated server (Hostinger KVM 4) running the backend
Docker	Every part of the backend runs in its own container
Load balancing	Nginx spreads traffic across 3 copies of the API
Cloudflare	HTTPS, attack protection, caching and DNS
Firewall	Only Cloudflare can reach the server
Event-driven system	RabbitMQ handles payments, orders and fulfilment separately
Monitoring	Grafana dashboards, logs, uptime checks and email alerts
Automatic deployment	Changes merged on GitHub are tested and deployed automatically
Modular codebase	Code organised into 8 business modules
Prod and dev environments	Two separate environments on the same server
Load testing	Tested with up to 30,000 shoppers at the same time

Environments

	ADDRESS	DATABASE
Production	api.hookapp.tech	New production cluster
Development	dev-api.hookapp.tech	Development cluster

Test results

TEST	RESULT
10,000 shoppers	99.6% of requests succeeded
30,000 shoppers in 20 seconds	98.7% succeeded, no server errors

Deliverables and access

Deliverables

DELIVERABLE	WHERE
Source code, all work merged	GitHub: `Hook-Deliveries/Hook-Backend`, branch `main`
Production environment	`https://api.hookapp.tech`
Development environment	`https://dev-api.hookapp.tech`
Configured server	Hostinger KVM 4, `153.92.222.181`
Automatic test and deploy pipeline	GitHub → Actions tab
Cloudflare setup (DNS, HTTPS, firewall, caching)	Cloudflare → `hookapp.tech`
Monitoring dashboards and email alerts	See the table on the right

What the Hook team needs for access

	ACCESS	HOW
1	Server access	Use the Windows SSH steps on page 3. Use the deploy account for project work and dashboard tunnels.
2	Dashboard logins	Grafana, Uptime Kuma and RabbitMQ logins, shared privately
3	GitHub	Write access to `Hook-Deliveries/Hook-Backend` to see tests and deploys
4	Cloudflare	Access to the `hookapp.tech` account for traffic and DNS
5	Alert emails	Sent to hook.africa2025@gmail.com; more addresses can be added

Dashboards to track the app

DASHBOARD	WHAT IT SHOWS	OPEN AT
Grafana	Traffic, errors, response times, server CPU, memory and disk, event queues and logs. Three dashboards: Hook Overview, Hook Application and VPS Server	`localhost:3000`
Uptime Kuma	Whether the API is up, response time and uptime history	`localhost:3001`
RabbitMQ (prod)	Payment, order and fulfilment event queues	`localhost:15673`
RabbitMQ (dev)	Development event queues	`localhost:15674`
Cloudflare Analytics	Visitors, requests, cache hits and blocked attacks	dash.cloudflare.com
GitHub Actions	Every test run and deployment	github.com

The dashboards are private. In a second Windows Command Prompt, open this tunnel, then visit the addresses above:

```
ssh -N -i "%USERPROFILE%\.ssh\id_ed25519" ^
  -L 3000:127.0.0.1:3000 ^
  -L 3001:127.0.0.1:3001 ^
  -L 15673:127.0.0.1:15672 ^
  -L 15674:127.0.0.1:15674 ^
  root@153.92.222.181
```

Keep the window open while you use the dashboards. Alerts arrive by email without opening anything.

Delivered by: Ogunseitan Gold Enoch · Date: 31 October 2026

Received for Hook by: _____ · Date: _____

HOOK BACKEND · OPERATIONS · NOVEMBER 2026

SSH access and environment updates

Use this runbook from the operator's Windows PC. Keep secrets in the server environment files only.

1. Connect from Windows Command Prompt

Open Command Prompt on the PC that holds her private key and run:

```
ssh -o IdentitiesOnly=yes -i "%USERPROFILE%\.ssh\id_ed25519" root@153.92.222.181
```

At the passphrase prompt, enter the key's passphrase. It is not the VPS password. If SSH asks to trust a host key, verify its fingerprint in Hostinger before accepting it.

2. Switch to the deployment account

```
sudo -iu deploy
whoami
```

The result should be `deploy`. Run project commands only after SSH connects and the remote Linux prompt appears.

3. Open the existing main environment file

```
cd /home/deploy/hook-backend/deploy
if [ -f .env ]; then nano .env; else echo ".env missing - stop"; fi
```

This checks that the real file exists before opening Nano. If it reports missing, do not create an empty file; confirm the project path first.

4. Change the requested variable

In Nano, press Ctrl+W and search for the exact variable name. Edit its existing line and keep only one assignment. Change only the approved value; leave unrelated settings alone.

```
FEATURE_FLAG=true
PUBLIC_SERVICE_URL=https://service.example
PRIVATE_API_KEY=(enter privately)
```

These are examples only. Use the exact value supplied by the project owner. Save with Ctrl+O, Enter, then Ctrl+X. Repeat in development only when that environment also needs the change.

Keep secret values on the server. Do not paste them into chat, share them on screen, or add them to GitHub.

Apply the change from the same deploy folder:

```
chmod 600 .env
docker compose up -d --force-recreate api-1 api-2 api-3 worker consumer-payments
consumer-orders consumer-fulfilment
docker compose ps
```

This restarts the app processes so they read the updated environment and can briefly interrupt requests.

Development is a separate checkout. Its expected file is `/home/deploy/hook-dev/deploy/.env`. Verify the directory and existing file before editing. Do not create it if missing.

Payment provider endpoints. Main callbacks: Paystack `https://api.hookapp.tech/api/v1/payments/paystack/callback`; Monnify `https://api.hookapp.tech/api/v1/payments/monnify/callback`. Register webhooks separately: `https://api.hookapp.tech/api/v1/webhooks/paystack` and `https://api.hookapp.tech/api/v1/webhooks/monnify`. Development uses `https://dev-api.hookapp.tech` with the same callback and webhook paths. Both environments currently use Paystack test credentials and Monnify sandbox mode; replace main credentials with live keys before accepting real payments. The app return URI is `hook://payments/return`.

HOOK BACKEND · OPERATIONS · NOVEMBER 2026

Environment variable change checklist

Use this process for any approved environment setting. Main and development have separate files and must be updated separately.

Before you edit

Confirm the exact variable name, approved value, which environment needs it, and which app processes read it. If the setting changes database, Redis, RabbitMQ, ports, or another service credential, coordinate all dependent services before editing.

For main, connect to the server and open the existing file:

```
cd /home/deploy/hook-backend/deploy
if [ -f .env ]; then nano .env; else echo ".env missing - stop"; fi
```

Press Ctrl+W to find the exact variable. Edit its existing line; do not add a duplicate. Keep one `NAME=value` assignment per setting.

```
VARIABLE_NAME=approved_value
FEATURE_FLAG=true
PUBLIC_SERVICE_URL=https://service.example
```

Examples are placeholders. For development, use `/home/deploy/hook-dev/deploy/.env` and change it only when requested. Never copy production secrets to development by default.

Keep secrets private

Do not run `cat .env`, paste values into chat, include them in screenshots, or commit them. Keep `.env` on the server; only variable names and empty placeholders belong in `.env.example`.

Save, apply, and verify

In Nano, save with Ctrl+O, Enter, then Ctrl+X. From that same deploy folder, run:

```
chmod 600 .env
docker compose up -d --force-recreate api-1 api-2 api-3 worker consumer-payments
consumer-orders consumer-fulfilment
docker compose ps
```

Check that the named setting exists without showing its value. Replace `VARIABLE_NAME` with the exact name:

```
grep -q '^VARIABLE_NAME=' .env && echo "setting present"
docker compose exec -T api-1 node -e 'const k="VARIABLE_NAME";
console.log(Object.hasOwn(process.env,k)?"loaded":"missing")'
```

Check the correct API health endpoint:

```
curl -sS -o /dev/null -w 'HTTP %{http_code}' https://api.hookapp.tech/health
```

For development, use `https://dev-api.hookapp.tech/health`. If a service is unhealthy, restore the previous value, recreate the affected app processes, and notify the project owner.

Container recreation applies environment changes; it does not publish code. Push source changes separately through the normal GitHub workflow.